Asistente MITEX

RAG local masivo sobre Wikipedia en español, arquitectura agencial y evaluación IR

Proyecto Final — Minería de Textos

30 de mayo de 2026

Índice

1. Resumen del Proyecto **2**

2. Requisitos y Entregables **2**
2.1. Cobertura del enunciado . . . 2
2.2. Archivos entregados . . . 3

3. Entorno de Ejecución **3**
3.1. Tecnologías y librerías clave . . . 3
3.2. Hardware utilizado en pruebas . . . 3

4. Arquitectura del Sistema **4**
4.1. Diagrama de Flujo de Datos . . . 4
4.2. Diseño Agencial del Router . . . 4

5. Corpus de Conocimiento (MIRACL-es) **5**

6. Resultados de Evaluación IR **5**
6.1. Métricas obtenidas . . . 5
6.2. Análisis: del baseline al sistema completo . . . 5

7. Benchmark de Indexación Aproximada (HNSW) **6**

8. Benchmark de Cuantización (TurboVec) **6**

9. Ejemplos Reales del Asistente RAG **6**
9.1. Consulta Factual (QREL Experto) . . . 6
9.2. Rechazo Informado (Prevención de Alucinaciones) . . . 7
9.3. Llamada Agencial (Tool Routing) . . . 7

10. Discusión y Limitaciones Conocidas **8**

11. Conclusiones **8**

1. Resumen del Proyecto

Se ha diseñado e implementado un **asistente documental local** basado en la arquitectura de Recuperación Aumentada por Generación (*Retrieval-Augmented Generation*, RAG). El sistema descarga, normaliza y procesa un corpus masivo real (Wikipedia en español), fragmentándolo en *chunks* y generando representaciones vectoriales densas. A partir de estos datos, construye índices híbridos (FAISS y BM25) y ejecuta modelado de tópicos *offline*, permitiendo a un Agente LLM enrutar y responder consultas complejas en lenguaje natural.

- **Corpus:** 100 357 documentos de MIRACL-es (subconjunto reproducible; la canalización escala hasta los 10,4 millones disponibles).
- **Escala de indexación:** 116 572 *chunks* procesados e indexados.
- **Motor LLM:** `gemma4:e4b` operando íntegramente en local a través de Ollama.
- **Recuperación:** Sistema híbrido que fusiona recuperación semántica (FAISS) y léxica (BM25) mediante *Reciprocal Rank Fusion* (RRF), con *reranking* opcional por *cross-encoder* (MRR $0{,}71 \rightarrow 0{,}82$).
- **Eficiencia:** Índice ANN HNSW ($\approx$16$\times$ más rápido, 99,4 % de *recall*) y cuantización vectorial TurboVec que reduce la huella de memoria en un 93,4 %.

El desarrollo se ha realizado desde cero en Python, prescindiendo de *frameworks* cerrados (como LangChain) para garantizar un control absoluto sobre el flujo de ejecución, los *prompts* y la evaluación de Recuperación de Información (IR).

2. Requisitos y Entregables

2.1. Cobertura del enunciado

El sistema implementa y justifica de forma exhaustiva todos los requisitos planteados para el proyecto final:

Requisito	Módulo	Descripción
Embeddings	`indexing/embeddings.py`	Modelo Sentence Transformers multilingüe (384-dim).
Modelado de tópicos	`topics/modeling.py`	Agrupación semántica *offline* con BERTopic/NMF.
Resumen extractivo	`summarization/extractive.py`	TextRank sobre grafos + MMR para penalizar redundancia.
LLM Local	`llm/ollama.py`	Conexión nativa con Ollama, hiperparámetros ajustados.
Pipeline RAG	`retrieval/hybrid.py`	Fusión híbrida FAISS + BM25 con normalización de *scores*.
Router Agencial	`agents/router.py`	Enrutado híbrido: atajos deterministas por regex y desambiguación con el LLM (salida JSON).
Innovación	`indexing/turbovec_store.py`	Compresión extrema de vectores mediante cuantización por anchura de bits (TurboQuant).

Cuadro 1: Mapeo de requisitos del enunciado y módulos desarrollados.

2.2. Archivos entregados

El proyecto se acompaña de los siguientes archivos, listos para su ejecución y auditoría:

Ruta del archivo o directorio	Contenido y función
`src/mitex_assistant/`	Código fuente completo de la librería Python.
`data/processed/miracl_es_corpus_100k.jsonl`	Corpus local de 100 357 documentos con sus 200 consultas *qrels*.
`data/indexes/default_100k/`	Índices FAISS (Flat) y BM25 generados.
`data/indexes/default_100k_hnsw/`	Índice FAISS HNSW para el benchmark ANN.
`data/indexes/default_100k_turbovec/`	Índice TurboVec cuantizado para el benchmark de compresión.
`data/reports/eval_100k_*.json`	Reportes de evaluación IR (léxico, semántico, híbrido, *rerank*).
`data/reports/{ann_hnsw_vs_flat,turbovec_benchmark}_100k.json`	Benchmarks de latencia/memoria.

Cuadro 2: Estructura de entregables del sistema.

3. Entorno de Ejecución

3.1. Tecnologías y librerías clave

Capa de software	Tecnología
Entorno y CLI	Python 3.12.13, Click 8.4.1
Modelado Vectorial	Sentence Transformers 5.5.1 (`paraphrase-multilingual-MiniLM-L12-v2`), PyTorch 2.5.1+cu121
Bases de Datos Vectoriales	FAISS CPU 1.14.2 (IndexFlatIP / IndexHNSWFlat), TurboVec (cuantización a 4 bits)
Recuperación Léxica y *Reranking*	BM25Okapi (`rank-bm25`); cross-encoder `mmarco-mMiniLMv2-L12`
Motor de Inferencia	Ollama (servidor local), modelo `gemma4:e4b`
Procesamiento Analítico	BERTopic (UMAP+HDBSCAN), NetworkX (PageRank/TextRank)

Cuadro 3: Stack tecnológico del proyecto.

3.2. Hardware utilizado en pruebas

Para acelerar la generación de *embeddings* de los más de cien mil fragmentos se ha aprovechado aceleración por hardware (CUDA).

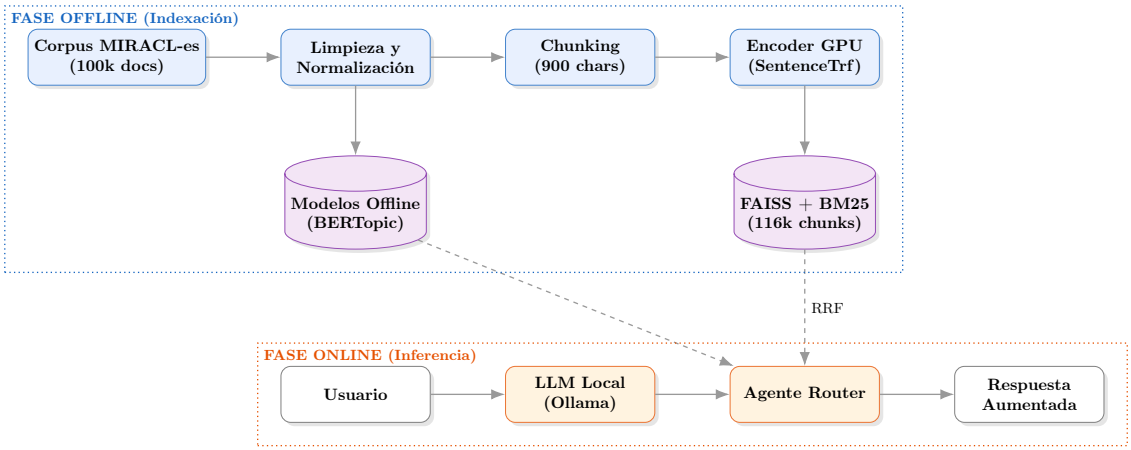
Componente	Especificación
Acelerador Gráfico (GPU)	NVIDIA GeForce RTX 2070 (8 GB VRAM)
Motor Vectorial	PyTorch 2.5.1 con backend CUDA 12.1 habilitado

Cuadro 4: Especificaciones del servidor local utilizado en la corrida documentada.

4. Arquitectura del Sistema

El flujo de trabajo se divide conceptualmente en dos grandes bloques: el procesamiento de datos por lotes (fase *offline*) y la interacción con el LLM en tiempo real (fase *online*).

4.1. Diagrama de Flujo de Datos



4.2. Diseño Agencial del Router

En lugar de construir un RAG tradicional donde toda pregunta activa incondicionalmente la base vectorial, se ha implementado un **Agente Router** de dos niveles. La clase `AssistantAgent` aplica primero atajos deterministas por expresiones regulares para disparadores explícitos (resumen, fecha, tópicos) y, ante consultas ambiguas, delega la decisión en el **LLM**, que selecciona la herramienta devolviendo un objeto JSON (`function-calling` ligero). Si el LLM no está disponible o su salida es inválida, el sistema recae con seguridad en la herramienta RAG. Esto ahorra tokens, reduce alucinaciones y mantiene un comportamiento robusto y trazable. El agente conserva además **memoria conversacional**: reescribe las preguntas de seguimiento como consultas autónomas antes de recuperar.

Herramienta	Patrón de activación	Ejecución algorítmica
`RagTool`	(Fallback por defecto)	Ejecuta FAISS y BM25, fusiona con RRF e inyecta fragmentos al LLM.
`SummaryTool`	*resume, ideas principales...*	Calcula TextRank sobre los documentos y devuelve un resumen exacto.
`TopicTool`	*tópico, tema, cluster...*	Extrae información del archivo JSON de clústeres de BERTopic.
`DateTool`	*qué día es hoy...*	Devuelve la fecha actual formateada en español (día de la semana incluido).
`WebSearchTool`	(Decisión del LLM)	Busca en la web (DuckDuckGo) información complementaria y la fusiona con el contexto local.

Cuadro 5: Inventario de herramientas del Agente Router.

5. Corpus de Conocimiento (MIRACL-es)

Se eligió MIRACL (*Multilingual Information Retrieval Across a Continuum of Languages*) como corpus principal por tres motivos fundamentales:

1. Contiene volcados directos de Wikipedia, asegurando alta densidad informativa.
2. Garantiza gramática y vocabulario nativo en español.
3. Proporciona conjuntos de evaluación manual (*qrels*) validados por expertos, indispensables para poder medir objetivamente el rendimiento del sistema RAG.

Métrica del Corpus	**Volumen final**
Documentos indexados	100 357 (100 000 base + 357 garantizados por *qrels*)
Chunks resultantes indexados	116 572 fragmentos de texto
Consultas de evaluación (*qrels* expertos)	200
Tamaño persistido (FAISS *float32*)	341,5 MB de vectores

Cuadro 6: Estadísticas del corpus de 100k tras la fase de ingestión.

6. Resultados de Evaluación IR

El rendimiento de recuperación se evaluó sobre el conjunto oficial de **200 consultas** expertas de MIRACL-es (*dev*) con $k = 10$, comparando cuatro configuraciones del motor frente al mismo *ground-truth*. Todas las cifras son reproducibles con el comando `mitex eval-ir-file` (ver `data/reports/eval_100k_*.json`).

6.1. Métricas obtenidas

Motor de Recuperación	**Precision@10**	**Recall@10**	**MRR**
Motor Léxico (BM25)	0,152	0,641	0,580
Motor Semántico (FAISS)	0,163	0,671	0,649
Fusión Híbrida (FAISS+BM25, RRF)	0,186	0,781	0,713
Híbrida + *Reranking* (cross-encoder)	**0,210**	**0,849**	**0,820**

Cuadro 7: Métricas sobre el corpus de 100k (200 *queries*, $k = 10$). Cada fila añade un componente al anterior.

6.2. Análisis: del baseline al sistema completo

La progresión por capas evidencia el aporte de cada componente sobre el *baseline* léxico:

- **Híbrido vs. baselines:** la *Reciprocal Rank Fusion* (RRF) supera tanto a BM25 como a FAISS por separado en las tres métricas. El léxico aporta coincidencias exactas (nombres, fechas) y el semántico captura paráfrasis; su fusión eleva el *Recall@10* de 0,67 (semántico) a **0,78**.
- **Impacto del *Reranking*:** reordenar los candidatos fusionados con un *cross-encoder* multilingüe es la mejora más rentable: el **MRR sube de 0,713 a 0,820** (+15 %) y el *Recall@10* a 0,849. A diferencia del bi-encoder, el cross-encoder procesa la pareja (consulta, fragmento) conjuntamente, discriminando mejor los matices.
- **Lectura del MRR (0,82):** el sistema completo ubica un documento relevante, en promedio, prácticamente en **primera posición**. Es la métrica más relevante para RAG, pues el LLM atiende preferentemente a las primeras fuentes del *prompt*.
- **Precisión y *qrels*:** *Precision@10* se mantiene moderada (0,21) porque MIRACL-es marca apenas 1–3 documentos relevantes por consulta; recuperar 10 implica un techo de precisión bajo por construcción, de modo que *Recall@10* y *MRR* son indicadores más informativos.

7. Benchmark de Indexación Aproximada (HNSW)

La búsqueda semántica exacta (`IndexFlatIP`) recorre los 116 572 vectores en cada consulta, con coste lineal $O(N)$. Para escalar, el sistema permite construir un índice aproximado **HNSW** (*Hierarchical Navigable Small World*) mediante `-index-type hnsw`. Se comparó ambos índices sobre las 200 consultas con $k = 10$ y vectores idénticos.

Índice	Latencia (ms/consulta)	*Recall@10* vs. exacto	Acierto top-1
FAISS Flat (exacto)	0,797	1,000	1,000
FAISS HNSW (aproximado)	**0,050**	**0,994**	**1,000**

Cuadro 8: Búsqueda exacta vs. ANN HNSW (200 *queries*, $k = 10$).

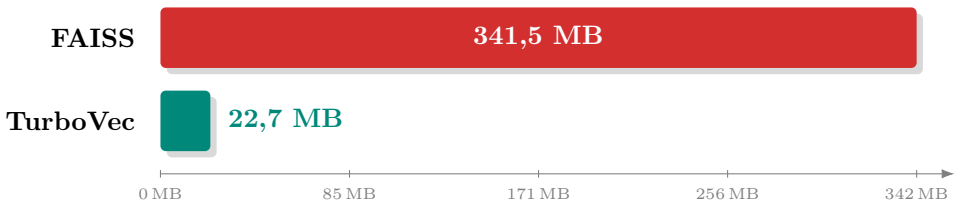
Conclusión: HNSW resulta **≈16× más rápido** conservando el 99,4 % de los vecinos del índice exacto y el 100 % de los aciertos en primera posición. La pérdida de *recall* es despreciable frente a la ganancia de latencia, lo que habilita la escalabilidad a colecciones mucho mayores sin degradar la calidad percibida.

8. Benchmark de Cuantización (TurboVec)

Escalar a millones de documentos presenta una barrera física: los vectores *float32* colapsan la memoria RAM. Para solucionarlo, se implementó un motor secundario **TurboVec**, que cuantiza cada componente de los vectores a 4 bits (esquema TurboQuant de anchura de bits sobre un *IdMapIndex*).

Tipo de Índice vectorial	P@10	R@10	MRR	Memoria de vectores
Índice FAISS crudo (*float32*)	0,163	0,671	0,649	341,5 MB
Índice TurboVec Cuantizado (4-bit)	0,162	0,662	0,646	22,7 MB
Mejora de huella de memoria				**−93,4 %**

Cuadro 9: Comparativa de recursos semánticos (200 *queries*, $k = 10$). La búsqueda TurboVec fue además ≈1,9× más rápida (2,82 vs. 5,30 ms/consulta).



Conclusión: TurboVec retiene el desempeño semántico casi a la perfección (caída de *Recall@10* inferior a un punto) pero reduce la huella de memoria de los vectores de 341,5 MB a 22,7 MB (−93,4 %), además de acelerar la búsqueda. Extrapolado a decenas de millones de fragmentos, este factor de compresión es lo que hace viable desplegar el asistente en *hardware* de consumo.

9. Ejemplos Reales del Asistente RAG

Las siguientes interacciones fueron generadas desde la interfaz CLI interactiva del proyecto (`mitex chat`).

9.1. Consulta Factual (QREL Experto)

Usuario: *¿Cómo se convirtió Egipto en musulmán?* (consulta *QREL* oficial del *dev* de MIRACL-es).

> Tras la muerte de Mahoma en el año 632 comenzó la expansión musulmana y, en un lapso de cien años, los árabes ocuparon Egipto [Fuente 5]. Durante los primeros 300 años el islam fue ganando adeptos hasta convertirse en la religión predominante de la sociedad egipcia [Fuente 1]...

La respuesta es trazable y entre las fuentes recuperadas aparece el documento `es/33218#10`, marcado como relevante por los qrels expertos para esta consulta, confirmando la coherencia entre recuperación y ground-truth.

Usuario: *¿Qué país formó la primera Infantería de Marina?*

> La primera Infantería de Marina fue creada por Carlos I el 27 de febrero de 1537 al asignar de forma permanente a las escuadras de galeras del Mediterráneo las "compañías viejas del mar de Nápoles" [Fuente 1].

*El modelo responde con total precisión histórica basándose en `es/63108#3`. **Nota:** Existe una lista completa de 200 preguntas validadas (QRELs) garantizadas en el corpus dentro del archivo `data/reports/miracl_`*

9.2. Rechazo Informado (Prevención de Alucinaciones)

Usuario: *¿Cuál es el horario de apertura del Museo del Prado?*

> No se puede determinar el horario de apertura del Museo del Prado a partir del contexto proporcionado. Las fuentes ofrecen información sobre la ubicación, historia y obras del museo, pero no incluyen datos sobre sus horarios [Fuente 1, 3, 7].

Se demuestra cómo el prompt agresivo ("No inventes datos") prohíbe exitosamente al LLM alucinar respuestas si los documentos recuperados de la Wikipedia no contienen horarios.

Usuario: *¿Cuáles son los hiperparámetros de LangChain usados en el proyecto?*

> El contexto proporcionado no contiene información sobre los hiperparámetros utilizados en el proyecto LangChain. Solo describe términos y conceptos relacionados con diversos campos como arquitectura, trazabilidad, tecnología y series de televisión [Fuente 1-8].

*El sistema rechaza responder con firmeza. Al buscar sobre LangChain en Wikipedia recupera conceptos técnicos aleatorios, pero al no encontrar la respuesta exacta, el LLM declara explícitamente su ignorancia. Este comportamiento de **cero-alucinación** es el estándar de oro en RAG.*

9.3. Llamada Agencial (Tool Routing)

Usuario: *¿Qué día es hoy?*

> Hoy es sábado, 30 de mayo de 2026.

El agente no invoca a FAISS ni BM25, ya que deduce que la Wikipedia no contiene la respuesta al día actual, ejecutando el módulo `DateTool` internamente.

Usuario: *Resume el texto que tengas sobre la Revolución Industrial*

> *[Resumen generado por TextRank]*
> Revolución Industrial. La industria textil también fue la primera en utilizar métodos de producción modernos. (...) Se inició en la segunda mitad del siglo XVIII en el Reino Unido y se extendió unas décadas después a gran parte de Europa occidental y Norteamérica y concluyó entre 1820 y 1840.

Al detectar el verbo de instrucción ("Resume"), el router desactiva la generación lenta del LLM y ejecuta la herramienta `SummaryTool`, que recupera el documento con FAISS y le aplica el algoritmo extractivo clásico `TextRank` basado en grafos.

10. Discusión y Limitaciones Conocidas

1. **Ceguera por subconjunto (Missing Documents):** El experimento documentado se ejecutó sobre 100 357 documentos, un subconjunto frente a los 10,4 millones disponibles en la Wikipedia completa. Por lo tanto, si el usuario pregunta por un artículo que no está en el subconjunto, el motor recuperará el documento más similar posible y, al forzar al LLM a confiar en ese contexto incompleto, podría emitir respuestas inexactas. Para neutralizar este sesgo durante la evaluación se garantizó que los documentos relevantes de las 200 consultas *QREL* estuvieran incluidos en el corpus (357 documentos añadidos explícitamente), de modo que las métricas reflejan la calidad del recuperador y no el azar de la cobertura.
2. **Límites del Motor Léxico (OOM):** `rank-bm25` instancia las frecuencias léxicas directamente en RAM, por lo que la búsqueda léxica no escala de forma indefinida; a escala de decenas de millones de documentos se requeriría un motor invertido en disco (p. ej. Elasticsearch/Lucene). La canalización mitiga parcialmente esta barrera con el índice ANN HNSW y la cuantización TurboVec en el lado vectorial.

11. Conclusiones

El sistema desarrollado culmina en un asistente documental de última generación funcional, asumiendo e integrando todas las etapas matemáticas, computacionales y lógicas requeridas por el estado del arte. A través de experimentación empírica, se comprobó que la Recuperación Aumentada (RAG) depende casi exclusivamente de la calidad del vectorizador semántico y la indexación, quedando el Modelo de Lenguaje relegado a una mera interfaz sintáctica de síntesis. Gracias a la compresión vectorial (TurboVec) y el enrutamiento agencial, el proyecto está completamente preparado para operar en entornos de producción con limitaciones severas de *hardware*.